

Labo 00 — Labo pratique : un tour de Linux par le terminal

Objectif : manipuler chaque notion de la théorie — processus, signaux, codes de sortie, permissions, environnement, flux, ports, archives — uniquement avec ce qu'Ubuntu 24.04 fournit d'origine. Aucun conteneur ici : tout ce que vous voyez ressortira, tel quel, dans les labos Docker.

Prérequis — Windows 10/11 avec WSL 2 et une distribution **Ubuntu 24.04**. Rien d'autre : ni Podman (labo 01), ni paquet supplémentaire. Ouvrez un terminal Ubuntu et restez-y.

Étape 0 — Où suis-je, et qui suis-je ?

```
head -n 2 /etc/os-release
uname -r
whoami
id
```

Observez `PRETTY_NAME="Ubuntu 24.04.x LTS"`, un noyau `6.6.87.2-microsoft-standard-WSL2` (le suffixe est la signature WSL), votre nom d'utilisateur, et une ligne `uid=1000(...) gid=1000(...)` `groupes=... 27(sudo) ...`.

Explication. Trois identités à ne plus confondre : la **distribution** (Ubuntu 24.04, l'espace utilisateur), le **noyau** (compilé par Microsoft pour WSL), et **vous** (UID 1000, membre du groupe `sudo`). Le noyau ne connaîtra de vous que ce nombre, 1000.

→ WINDOWS / WSL

Si `uname -r` n'affiche pas `-microsoft-standard-WSL2`, vous n'êtes pas dans WSL 2 (`wsl --version` et `wsl --list --verbose` côté PowerShell pour vérifier). Toute la série de labos suppose WSL 2.

Étape 1 — Le noyau, l'espace utilisateur, et la frontière

```
cat /proc/version
type ls
type cd
which cat
```

Observez : la version du noyau en toutes lettres ; `ls` is `/usr/bin/ls` (un programme, un fichier sur disque) ; `cd` is `a shell builtin` (pas un programme : une fonction interne du shell) ; `/usr/bin/cat`.

Explication. Tout ce que vous tapez est soit un **programme** de l'espace utilisateur (un fichier exécutable quelque part), soit une commande interne du shell. Aucun des deux ne touche le matériel : ils passent par les appels système du noyau. `cd` est interne pour une raison précise : changer de répertoire est un attribut *du processus shell lui-même* — un programme externe changerait le répertoire de son propre processus, puis mourrait, sans effet sur vous.

→ LINUX

`type` interroge le shell (« que ferais-tu de ce mot ? ») ; `which` cherche seulement dans le `PATH`. En cas de doute sur une commande qui « ment » (alias, fonction), `type` dit toujours la vérité.

Étape 2 — Un seul arbre, des montages

```
ls /  
findmnt /  
df -h /  
ls /mnt/c/Windows 2>/dev/null | head -n 3
```

Observez la racine unique (`bin boot dev etc home ... proc ... tmp usr var`), la ligne `findmnt` du type `/ /dev/sdc ext4 rw,relatime,...`, un `df` de l'ordre de `1007G` (la taille *virtuelle* du disque WSL), et — c'est du Windows vu depuis Linux — le contenu de `C:\Windows`.

```
findmnt -t proc  
ls /proc | head -n 8  
grep MemTotal /proc/meminfo
```

Observez `/proc proc proc rw,relatime` : un système de fichiers de type `proc`, sans disque derrière. Le `ls` liste des **nombres** — un par processus vivant — et `MemTotal` vient directement du noyau.

Explication. Pas de lecteurs `C: / D:` : tout est accroché au même arbre par des **montages**. Le disque Linux fournit `/`, le disque Windows est monté sur `/mnt/c`, et le noyau lui-même est monté sur `/proc` — un répertoire dont les fichiers sont fabriqués à la volée à chaque lecture. Les images Docker (labo 02) et les volumes (labo 06) ne feront qu'ajouter des montages à cet arbre.

→ WINDOWS / WSL

`/mnt/c` traverse une frontière Windows ↔ Linux : c'est **lent**. Un projet qu'on compile ou des images qu'on stocke doivent vivre côté Linux (`/home/...`), pas dans `/mnt/c/Users/...`. Réflexe à prendre dès maintenant.

Étape 3 — Les processus : PID, parent, /proc

```
ps  
echo $$  
ps -p 1 -o pid,comm
```

Observez : `ps` presque vide (votre `bash`, le `ps` lui-même) ; le PID de votre shell (`echo $$`) ; et le processus 1 : `systemd`.

Lancez maintenant un processus qui dure, en arrière-plan :

```
sleep 300 &  
ps -o pid,ppid,stat,cmd
```

Observez une ligne `sleep 300` dont le **PPID est le PID de votre bash** : vous venez de voir une filiation.

```
PID  PPID  STAT  CMD
2363  2362  S      bash
2419  2363  S      sleep 300
2420  2363  R      ps -o pid,ppid,stat,cmd
```

Allez voir ce processus dans `/proc` (remplacez `2419` par votre PID) :

```
head -n 3 /proc/2419/status
tr '\0' ' ' < /proc/2419/cmdline; echo
ls -l /proc/2419/exe
```

Observez `Name: sleep`, `State: S (sleeping)`, la ligne de commande exacte, et un lien `exe -> /usr/bin/sleep`.

Explication. `ps` n'a rien de magique : il lit `/proc`. Tout ce que Docker vous montrera plus tard (`podman top`, `podman inspect`) vient de là aussi. `STAT S` signifie *sleeping* — en attente ; `R`, *running*.

Étape 4 — Signaux et codes de sortie

Le `sleep` tourne toujours. Congédiez-le poliment :

```
kill 2419          # votre PID à vous
ps -o pid,cmd | grep "[s]leep 300" || echo "plus de processus sleep"
```

Observez `Terminated` (affiché par le shell) puis `plus de processus sleep` : `kill` sans option envoie `SIGTERM`, et `sleep` obéit.

Recommencez, brutalement :

```
sleep 300 &
kill -9 %1
```

Observez cette fois `Killed` : `SIGKILL` n'a rien demandé. (`%1` désigne le *job* n°1 du shell — pratique pour ne pas chercher le PID.)

Maintenant, la collecte des codes de sortie :

```
true; echo $?
false; echo $?
ls /nexiste-pas; echo $?
commande-inconnue; echo $?
bash -c 'kill -9 $$'; echo $?
```

Observez, dans l'ordre : `0`, `1`, `2` (après le message d'erreur de `ls`), `127` (après `command not found`), et `137` (après `Killed`).

Explication. `0` = succès, le reste = échec, et `128 + n` = mort par le signal *n* : `137 = 128 + 9` = tué par `SIGKILL`. Ces cinq nombres sont exactement ce que `podman ps` affichera dans sa colonne `Exited (...)` au labo 03 — apprenez à les lire ici, où tout est simple.

À RETENIR

L'escalade civilisée : `kill` (SIGTERM, l'application peut ranger), attendre, puis seulement `kill -9` (SIGKILL, le noyau efface). `docker stop` applique ce protocole automatiquement : SIGTERM, 10 secondes de grâce, SIGKILL.

Étape 5 — L'environnement et le PATH

```
echo $HOME
env | wc -l
env | grep -E '^(HOME|PATH|LANG)='
```

Observez votre environnement : quelques dizaines de variables, dont `HOME=/home/<vous>` et un `PATH` qui, sur WSL, contient aussi des chemins Windows (`/mnt/c/Windows/system32` ...).

L'expérience décisive — une variable de shell n'est **pas** une variable d'environnement :

```
MSG=coucou
echo $MSG
bash -c 'echo fils voit: [$MSG]'
export MSG
bash -c 'echo fils voit: [$MSG]'
```

Observez : `coucou`, puis `fils voit: []` (vide !), puis, après `export`, `fils voit: [coucou]`.

Explication. Chaque processus enfant reçoit une **copie** de l'environnement du parent, figée au lancement. Avant `export`, `MSG` n'existait que dans votre shell. C'est ce mécanisme exact que `podman run -e MSG=coucou` utilisera au labo 08 pour configurer vos applications.

Ensuite, le `PATH` :

```
mkdir -p ~/labo0/utils
printf '#!/bin/bash\neco "outil maison : ok"\n' > ~/labo0/utils/monoutil
chmod +x ~/labo0/utils/monoutil
monoutil; echo $?
export PATH="$HOME/labo0/utils:$PATH"
monoutil
```

Observez d'abord `command not found` et `127`, puis, une fois le répertoire ajouté au `PATH`, `outil maison : ok`.

Explication. Le shell ne « connaît » aucune commande : il cherche un exécutable de ce nom dans les répertoires du `PATH`, dans l'ordre, et s'arrête au premier trouvé. (Ce `PATH` modifié ne vaut que pour ce shell ; permanent = une ligne dans `~/.bashrc`.)

Étape 6 — Trois flux : redirections et pipes

```
cd ~/labo0
echo "première ligne" > notes.txt
echo "deuxième ligne" >> notes.txt
cat notes.txt
```

Observez : `>` crée (ou écrase !), `>>` ajoute.

Séparez maintenant les deux flux de sortie :

```
ls notes.txt /nexiste-pas > sortie.txt 2> erreurs.txt
cat sortie.txt
cat erreurs.txt
```

Observez : l'écran est resté muet pendant le `ls` ; `sortie.txt` contient `notes.txt`, `erreurs.txt` contient `ls: cannot access '/nexiste-pas': No such file or directory`.

```
ls notes.txt /nexiste-pas > tout.txt 2>&1
cat tout.txt
```

Observez les deux lignes réunies : `2>&1` branche le flux d'erreur (2) là où pointe la sortie (1).

Enfin, les pipes :

```
ps -ef | wc -l
ps -ef | grep "[b]ash" | head -n 3
```

Observez le nombre de processus du système, puis vos shells — sans fichier intermédiaire : la sortie de chaque commande alimente l'entrée de la suivante.

→ LINUX / SHELL

L'astuce `grep "[b]ash"` : les crochets forment une expression régulière qui matche `bash` ... mais la ligne du `grep` lui-même contient `[b]ash`, qui ne se matche pas. Sans cela, `grep` se trouverait toujours lui-même. Vous verrez ce motif dans tous les labos.

Étape 7 — Permissions : lire un `ls -l`

```
ls -l notes.txt
stat -c "%U %G %a %n" notes.txt
```

Observez `-rw-r--r-- 1 <vous> <vous> 30 ... notes.txt` et sa forme numérique `644` : propriétaire `rw` (6), groupe `r` (4), autres `r` (4).

Créez un script et essayez de l'exécuter :

```
printf '#!/bin/bash\nnecho "Bonjour, je suis le processus $$"\n' > salut.sh
./salut.sh; echo $?
chmod +x salut.sh
ls -l salut.sh
./salut.sh
```

Observez : `Permission denied` et le code **126** (trouvé mais non exécutable) ; puis, après `chmod +x`, `-rwxr-xr-x` et le script qui s'exécute — avec un PID différent à chaque lancement.

Et la frontière root :

```
cat /etc/shadow; echo $?
ls -l /etc/shadow
sudo head -n 1 /etc/shadow
```

Observez `Permission denied` (code 1), la ligne `-rw-r----- 1 root shadow ...` qui l'explique (vous n'êtes ni `root` ni du groupe `shadow`), puis, via `sudo`, la première ligne `root:!:...` (`*` ou `!` : compte verrouillé, aucun mot de passe accepté).

Explication. Le noyau compare l'UID du processus aux trois triplets `rwX` et applique le premier qui vous concerne. `sudo` ne « contourne » rien : il lance le processus avec l'UID 0, auquel le noyau ne refuse rien. Au labo 06, quand un conteneur écrira dans un volume des fichiers appartenant à un UID inattendu, c'est cette grille de lecture qu'il vous faudra.

Étape 8 — Un serveur, un port, un client

Ubuntu 24.04 embarque Python : votre premier serveur HTTP en une ligne.

```
echo "<h1>Bonjour depuis mon serveur</h1>" > index.html
python3 -m http.server 8080 &
curl -s http://localhost:8080/index.html
```

Observez votre HTML renvoyé par HTTP : `<h1>Bonjour depuis mon serveur</h1>`.

```
curl -si http://localhost:8080/index.html | head -n 4
ss -tlnp | grep 8080
```

Observez la réponse HTTP complète (`HTTP/1.0 200 OK`, `Server: SimpleHTTP/0.6 Python/3.12.3`, `Content-type: text/html`) et la ligne d'écoute :

```
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(("python3",pid=2788,fd=3))
```

`0.0.0.0:8080` : le processus `python3` écoute sur **toutes** les interfaces, port 8080.

→ **WINDOWS / WSL**

Ouvrez un navigateur **Windows** sur `http://localhost:8080` : la page s'affiche. WSL 2 relaie automatiquement `localhost` de Windows vers Ubuntu. C'est ce relais qui, au labo 07, vous permettra de tester vos conteneurs depuis un navigateur Windows.

Essayez maintenant un port privilégié :

```
python3 -m http.server 80
```

Observez l'échec : `PermissionError: [Errno 13] Permission denied`. Le port 80 est sous le seuil 1024, réservé à root — et vous êtes UID 1000. Podman rootless héritera de la même limite.

Éteignez le serveur et vérifiez :

```
kill %1
curl -s --max-time 2 http://localhost:8080/; echo $?
```

Observez le code **7** de `curl` : *connection refused* — plus personne n'écoute.

Étape 9 — Archiver : `tar`, l'ancêtre des images

```
mkdir -p mon-app/config
echo "app.port=8080" > mon-app/config/app.properties
echo "binaire factice" > mon-app/app.bin
tar -czf mon-app.tar.gz mon-app
ls -lh mon-app.tar.gz
file mon-app.tar.gz
```

Observez une archive de quelques centaines d'octets, identifiée `gzip compressed data`.

```
tar -tf mon-app.tar.gz
mkdir -p /tmp/restauration
tar -xzf mon-app.tar.gz -C /tmp/restauration
cat /tmp/restauration/mon-app/config/app.properties
```

Observez la liste du contenu (`-t` = *test/list*), puis l'extraction ailleurs (`-C`) et le fichier restauré à l'identique : `app.port=8080`.

Explication. `tar` (*tape archive*, 1979) met une arborescence complète — chemins, permissions, propriétaires — dans un seul fichier. Retenez-le bien : une **couche** d'image Docker est littéralement une archive tar, et `podman save` (labo 02) vous produira un tar de tars. Rien de neuf sous le soleil.

Nettoyage

Vérifiez qu'aucun processus du labo ne traîne, puis supprimez les fichiers :

```
ps -o pid,cmd | grep -E "[s]leep|[h]ttp.server" || echo "rien à tuer"
rm -r ~/labo0
rm -r /tmp/restauration
```

Le `PATH` modifié et la variable `MSG` disparaîtront avec ce shell : fermez le terminal. (Rien n'a été installé : il n'y a rien à désinstaller.)

Ce que vous devez pouvoir affirmer maintenant

- Mon noyau est `...-microsoft-standard-WSL2` ; ma distribution est Ubuntu 24.04 ; je suis l'UID 1000.
- Un processus a un PID et un parent ; je l'ai vu naître (`&`), vécu (`/proc/<pid>/`), et mourir (`kill`).
- `kill` envoie SIGTERM (négociable), `kill -9` SIGKILL (non négociable) ; un processus tué par SIGKILL sort en `137` = `128 + 9`.
- `$?` vaut `0` en cas de succès ; `126` = non exécutable, `127` = introuvable dans le `PATH`.
- Une variable n'atteint les processus enfants qu'après `export` — et jamais les processus déjà lancés.
- `>` capture stdout, `2>` stderr, `2>&1` les fusionne, `|` enchaîne les processus.
- `-rw-r-----` se lit en trois triplets ; le noyau compare des UID, et `root` (UID 0) ignore la grille.
- `ss -tlnp` me dit qui écoute sur quel port ; `0.0.0.0` = toutes les interfaces ; `< 1024` = root seulement ; `curl` teste le tout.
- Un montage accroche un système de fichiers à l'arbre unique ; `/proc` n'a pas de disque ; `tar` emballe une arborescence — les images Docker feront pareil.